1. **Which transformer architecture would be most appropriate for building a text classification model in PyTorch?** 1 point

- Decoder-only, because it can generate the classification label
- Encoder-decoder, because it can transform the input into a classification output
- Encoder-only, because encoders are designed for understanding tasks
- Any architecture works equally well for classification tasks

2. **In the attention mechanism, what do the Query (Q), Key (K), and Value (V) representations represent for each token?** 1 point

- Q represents how this token appears to others, K represents what this token searches for, V represents the information this token provides
- Q represents what this token searches for, K represents how this token appears to others, V represents the information this token provides
- Q represents the information this token provides, K represents how this token appears to others, V represents what this token searches for
- Q represents what this token searches for, K represents the information this token provides, V represents how this token appears to others

3. **What is the correct sequence of operations for calculating attention weights?** 1 point

- Softmax the queries, scale the result, then take dot product with keys
- Dot product of queries and keys, scale the result, then apply softmax
- Scale the queries, take dot product with keys, then apply softmax
- Dot product of queries and values, scale the result, then apply softmax

4. **When using nn.MultiheadAttention for self-attention in an encoder, how should you pass the input to compute queries, keys, and values?** 1 point

- Pass the input once and let PyTorch automatically create Q, K, and V
- Pass three different transformed versions of the input
- Pass the same input three times for Q, K, and V
- Pass the input twice - once for keys and values, and separately for queries

5. **When creating an nn.TransformerEncoderLayer, what does the d_model parameter specify?** 1 point

- The number of attention heads in the layer
- The number of encoder layers to stack
- The dimensionality of the token embeddings
- The size of the feed-forward network

6. **When building a decoder in PyTorch, how can you apply causal masking to prevent tokens from attending to future positions?** 1 point

- Set bidirectional=False in the attention layer
- Set is_causal=True or pass a causal mask
- Call nn.CausalAttention instead of nn.MultiheadAttention
- Set decoder_mode=True in the layer configuration

7. **In PyTorch, how can you build a decoder-only model using existing encoder components?** 1 point

- You must use nn.TransformerDecoderLayer, encoder layers cannot function as decoders
- Reuse nn.TransformerEncoderLayer or nn.TransformerEncoder and pass a causal mask
- Create a custom decoder class from scratch, PyTorch doesn't support decoder-only models
- Use nn.TransformerEncoder with reverse=True to make it generate text

8. **In an encoder-decoder Transformer used for machine translation, cross-attention allows the decoder to:** 1 point

- Process its own outputs using queries, keys, and values all generated within the decoder
- Use queries from the decoder to attend to keys and values from the encoder
- Share its attention weights with the encoder to maintain consistency across both components
- Apply bidirectional attention to future target tokens while generating one word at a time

9. **When calling a nn.TransformerDecoderLayer in PyTorch, what is passed as the memory parameter?** 1 point

- The target sequence that the decoder is generating
- The previous decoder layer's output
- The encoder's output representation of the source sequence
- The causal mask for preventing attention to future tokens

10. **How does a nn.TransformerDecoderLayer differ structurally from a nn.TransformerEncoderLayer?** 1 point

- Decoder layers use more feed-forward sublayers than encoder layers
- Decoder layers have two attention blocks while encoder layers have only one
- Decoder layers can only process one token at a time while encoder layers process full sequences
- Decoder layers don't use residual connections while encoder layers do